

07 — Trees

DSA in Python — Interview Prep

Trees — Tutorial

Covers the 10 problems solved in this folder, plus the shared `tree_node.py` (a `TreeNode` class and `build_tree/to_level_order/find_node` helpers used by every test harness here).

Almost every tree problem is a variation on the same shape: define what a function should compute *for a single node*, assuming its children already know the answer for themselves (recursion handles the "assuming" part for free), then combine those child answers at the current node. The 10 problems below are really 10 different answers to "what do you combine, and how."

The patterns

1. Depth-first traversal — recursive vs. explicit stack (01)

Recursive inorder (`inorder(left); visit(node); inorder(right)`) is one line, but it's secretly using the call stack as storage. Making that storage explicit: push every node down a left-child chain onto a stack (mirrors "go as far left as possible"), pop and visit one, then repeat the same left-chain push starting from its right child. Worth doing once by hand, since interviewers sometimes specifically ask for the iterative version to check this isn't just memorized.

2. Simplest recursion — compute one value per subtree (02)

`max_depth` is the template every later pattern builds on: base case returns a trivial value for an empty subtree (0), recursive case combines the two children's answers with the current node's own contribution (`1 + max(...)`). Nothing more complex than this shape is needed for most "compute a number about this tree" problems.

3. Rebuilding instead of computing (03)

`invert_tree` uses the exact same recursive shape as pattern 2, but returns a (modified) *node* instead of a number. Swap `root.left` and `root.right` using the recursively-inverted versions of each — since Python evaluates both call results before assigning, there's no risk of overwriting one child before the other has been inverted.

4. Breadth-first traversal (04)

Every pattern so far is depth-first. Level order needs a queue instead of a stack/recursion, and the one non-obvious detail is snapshotting `len(queue)` *before* the inner loop that processes one level — otherwise the children being pushed during that loop would grow the count being iterated over and blur where one level ends and the next begins.

5. Structural comparison across two subtrees (05)

`is_symmetric` compares two subtrees against each other rather than computing one value for one tree. The key subtlety: mirror symmetry means comparing `a.left` to `b.right` (outer-to-outer, inner-to-inner swapped), not `a.left` to `b.left` (which would just check the two sides are identical copies, a different and easier

problem).

6. Computing a global answer during a per-node pass (06)

`diameter_of_binary_tree` needs the *best* value seen at *any* node in the tree, not just the value at the root. Rather than a separate traversal to find the max, piggyback on the height computation (pattern 2's shape) with a `nonlocal` variable updated as a side effect on every call — each call still returns the height its caller needs, while also recording "the diameter through me" into the shared running maximum.

7. Early-exit sentinel to avoid recomputation (07)

A naive "is this balanced" check recomputes height from scratch at every node while separately checking balance, costing $O(n^2)$ on a skewed tree. `is_balanced` reuses pattern 2's height function directly but returns -1 as a sentinel the instant a subtree below is found unbalanced, and every caller checks for and immediately re-propagates that -1 without doing further work. Each node's height is then computed exactly once — $O(n)$ total — with early termination once imbalance is already known anywhere below.

8. Threading a valid range through the recursion (08)

The naive "check each node against its immediate parent" approach for BST validation is wrong — it misses violations from higher ancestors (see the worked counterexample in `08_validate_bst.py`). The fix: pass a (`low`, `high`) range *down* through the recursion, tightened by every ancestor so far (left children tighten the upper bound to the parent's value, right children tighten the lower bound), not just the immediate parent.

9. Exploiting BST ordering to skip a subtree entirely (09)

`lowest_common_ancestor_bst` doesn't need to explore both children at any step. Comparing p and q's values to the current node's value directly says which single subtree could possibly contain the LCA — both smaller means go left, both larger means go right, and the moment they're not both on the same side, the current node itself is the split point. One walk down, no recursion into both branches.

10. General-tree LCA without ordering (10)

Without a BST's ordering property, there's no shortcut for which side to search, so

`lowest_common_ancestor_binary_tree` must explore both. A call returns early the moment it hits p or q itself (a node can be its own ancestor). If both the left and right recursive calls come back non-None, p and q were found on opposite sides, so the *current* node is where their paths converge; if only one side is non-None, both targets are in that one subtree, so that result just passes upward unchanged.

Complexity summary

#	Problem	Time	Extra space
01	<code>binary_tree_inorder_traversal</code>	$O(n)$	$O(h)$
02	<code>maximum_depth_of_binary_tree</code>	$O(n)$	$O(h)$
03	<code>invert_binary_tree</code>	$O(n)$	$O(h)$
04	<code>level_order_traversal</code>	$O(n)$	$O(n)$
05	<code>symmetric_tree</code>	$O(n)$	$O(h)$
06	<code>diameter_of_binary_tree</code>	$O(n)$	$O(h)$
07	<code>balanced_binary_tree</code>	$O(n)$	$O(h)$

#	Problem	Time	Extra space
08	validate_bst	$O(n)$	$O(h)$
09	lowest_common_ancestor_bst	$O(h)$	$O(1)$
10	lowest_common_ancestor_binary_tree	$O(n)$	$O(h)$

h is the tree's height — $O(\log n)$ for a balanced tree, $O(n)$ for a completely skewed one (a "linked list in disguise"), which is why interviewers often ask "what's the worst case?" specifically for trees: a "balanced" complexity claim can quietly assume balance that isn't guaranteed. 09 is the one problem here that's genuinely faster than $O(n)$, and only because it exploits the BST property to avoid visiting most of the tree at all.

Common interview follow-ups to be ready for

- `binary_tree_inorder_traversal`: "can you also do preorder and postorder iteratively?" — preorder is a small tweak of this same stack; postorder is the hardest of the three (commonly done by reversing a modified preorder, or tracking a "last visited" node).
- `diameter_of_binary_tree` / `balanced_binary_tree`: "why not compute height separately and diameter/balance separately?" — be ready to state the complexity cost explicitly: separate passes risk $O(n^2)$ on skewed trees, versus $O(n)$ for the combined single-pass version.
- `validate_bst`: "what if the tree has duplicate values?" — this implementation uses strict `</>`, so duplicates anywhere are correctly rejected; confirm that's the intended rule before coding, since some variants allow equal values on one side.
- `lowest_common_ancestor_bst`: "what if the tree weren't a BST?" — degrades to problem 10's approach; be ready to explain concretely *why* the BST-specific shortcut becomes invalid without ordering.
- `lowest_common_ancestor_binary_tree`: "what if p or q might not actually be in the tree?" — this implementation assumes both exist (per the problem statement); discuss what changes if that's not guaranteed (a separate existence check first, or returning a sentinel).
- General: "what if the tree is extremely unbalanced / very deep?" — every recursive solution here risks a stack overflow on a sufficiently skewed tree; know that an iterative rewrite (explicit stack, as in 01) is the fix, and be ready to sketch one live.

Self-check before moving on

You should be able to, without looking at the code:

- Write the iterative inorder traversal from memory.
- Explain why level order needs `len(queue)` snapshotted before the inner loop.
- State the "combine child answers, propagate a global side effect" shape shared by diameter and balanced-tree, and why it avoids the naive $O(n^2)$.
- Explain why BST validation must track a range from every ancestor, not just the immediate parent, with the specific counterexample that breaks a parent-only check.
- State the one-sentence difference between the BST and general-tree LCA algorithms, and why the BST version doesn't need both subtrees.

If any of those aren't fluent yet, this topic isn't "done" per the README's definition — revisit before moving to `08_heaps_priority_queue`.

Programs

01. Binary Tree Inorder Traversal

01_binary_tree_inorder_traversal.py

Explanation

Problem: Given the root of a binary tree, return its inorder traversal (left, node, right) as a list of values. Do it iteratively (no recursion), to practice the technique explicitly rather than relying on the call stack to do it implicitly.

Source : LeetCode 94 - Binary Tree Inorder Traversal

Example:

Input: 1

\

2

/

3

Output: [1, 3, 2]

Idea: recursive inorder is trivial (`inorder(left); visit(node); inorder(right)`), but that's really using the **call stack** as an implicit stack. Making it explicit: push every left-child down a chain onto a stack first (mirrors "go as far left as possible"), then pop one, visit it, and repeat the same "push everything on the left chain" process starting from its right child. This same explicit-stack-instead-of-recursion swap generalizes to preorder and postorder too.

Code

```
from tree_node import build_tree

def inorder_traversal(root):
    result = []
    stack = []
    curr = root

    while curr or stack:
        while curr:
            stack.append(curr)
            curr = curr.left
        curr = stack.pop()
        result.append(curr.val)
        curr = curr.right

    return result

if __name__ == "__main__":
    tests = [
```

```

        ([1, None, 2, 3], [1, 3, 2]),
        ([], []),
        ([1], [1]),
        ([1, 2, 3, 4, 5], [4, 2, 5, 1, 3]),
    ]

    for i, (inp, expected) in enumerate(tests, 1):
        got = inorder_traversal(build_tree(inp))
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")

```

02. Maximum Depth Of Binary Tree

02_maximum_depth_of_binary_tree.py

Explanation

Problem: Given the root of a binary tree, return its maximum depth (the number of nodes along the longest path from root to a leaf).

Source : LeetCode 104 - Maximum Depth of Binary Tree

Example:

Input: 3

/ \

9 20

/ \

15 7

Output: 3

Idea: the simplest possible tree recursion, and the template every later "compute something about a subtree" problem in this folder builds on. A tree's depth is 1 (for the current node) plus whichever child subtree is deeper - recursion handles "how deep is this subtree" identically at every level, bottoming out at an empty subtree having depth 0.

Code

```

from tree_node import build_tree

def max_depth(root):
    if not root:
        return 0
    return 1 + max(max_depth(root.left), max_depth(root.right))

if __name__ == "__main__":
    tests = [
        ([3, 9, 20, None, None, 15, 7], 3),
        ([], 0),
        ([1], 1),
        ([1, 2], 2),
    ]

    for i, (inp, expected) in enumerate(tests, 1):
        got = max_depth(build_tree(inp))
        status = "PASS" if got == expected else "FAIL"

```

```
print(f"Test {i}: {status} (got {got}, expected {expected})")
```

03. Invert Binary Tree

03_invert_binary_tree.py

Explanation

Problem: Given the root of a binary tree, invert it (mirror every subtree - swap left and right children, recursively) and return the root.

Source : LeetCode 226 - Invert Binary Tree

Example:

Input: 4 Output: 4

/ \ \

2 7 7 2

/ \ \ / \ \

1 3 6 9 9 6 3 1

Idea: same recursive shape as 02, but instead of **computing** a value from the subtrees, it **rebuilds** the tree from inverted subtrees.

Swap root.left and root.right using the recursively-inverted versions of each - Python evaluates both `invert_tree(...)` calls before doing the assignment, so there's no need for a temporary variable to avoid overwriting one child before the other is inverted.

Code

```
from tree_node import build_tree, to_level_order

def invert_tree(root):
    if not root:
        return None
    root.left, root.right = invert_tree(root.right), invert_tree(root.left)
    return root

if __name__ == "__main__":
    tests = [
        ([4, 2, 7, 1, 3, 6, 9], [4, 7, 2, 9, 6, 3, 1]),
        ([2, 1, 3], [2, 3, 1]),
        ([], []),
    ]

    for i, (inp, expected) in enumerate(tests, 1):
        got = to_level_order(invert_tree(build_tree(inp)))
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")
```

04. Level Order Traversal

04_level_order_traversal.py

Explanation

Problem: Given the root of a binary tree, return its level-order traversal - a list of lists, one per depth level, left to right.

Source : LeetCode 102 - Binary Tree Level Order Traversal

Example:

Input: 3

/ \

9 20

/ \

15 7

Output: [[3], [9, 20], [15, 7]]

Idea: every other traversal in this folder so far is depth-first (recursion or an explicit stack). Level order is breadth-first, which needs a queue instead - process one full level of the queue at a time by snapshotting `len(queue)` *before* the inner loop starts (since pushing that level's children during the loop would otherwise grow the count being iterated over and blur level boundaries).

Code

```
from collections import deque

from tree_node import build_tree


def level_order(root):
    if not root:
        return []

    result = []
    queue = deque([root])

    while queue:
        level = []
        for _ in range(len(queue)):
            node = queue.popleft()
            level.append(node.val)
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
        result.append(level)

    return result


if __name__ == "__main__":
    tests = [
        ([3, 9, 20, None, None, 15, 7], [[3], [9, 20], [15, 7]]),
        ([1], [[1]]),
        ([], []),
    ]

    for i, (inp, expected) in enumerate(tests, 1):
        got = level_order(build_tree(inp))
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")
```

05. Symmetric Tree

05_symmetric_tree.py

Explanation

Problem: Given the root of a binary tree, check whether it is a mirror of itself (symmetric around its center).

Source : LeetCode 101 - Symmetric Tree

Example:

Input: 1

/ \

2 2

/ \ \

3 4 4 3

Output: True

Idea: symmetry means the left subtree is a **mirror image** of the right subtree, not just an equal copy. Compare two subtrees at once, checking outer-with-outer and inner-with-inner: **a.left** against **b.right**, and **a.right** against **b.left**. This is structurally different from a plain "are these two trees identical" check (which would compare **a.left** to **b.left**) - the swap is the entire point.

Code

```
from tree_node import build_tree

def is_symmetric(root):
    def is_mirror(a, b):
        if not a and not b:
            return True
        if not a or not b:
            return False
        return (
            a.val == b.val
            and is_mirror(a.left, b.right)
            and is_mirror(a.right, b.left)
        )

    return is_mirror(root.left, root.right) if root else True

if __name__ == "__main__":
    tests = [
        ([1, 2, 2, 3, 4, 4, 3], True),
        ([1, 2, 2, None, 3, None, 3], False),
        ([], True),
        ([1], True),
    ]

    for i, (inp, expected) in enumerate(tests, 1):
        got = is_symmetric(build_tree(inp))
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")
```

06. Diameter Of Binary Tree

06_diameter_of_binary_tree.py

Explanation

Problem: Given the root of a binary tree, return the length (number of edges) of the longest path between any two nodes. The path does not need to pass through the root.

Source : LeetCode 543 - Diameter of Binary Tree

Example:

Input: 1

/ \

2 3

/ \

4 5

Output: 3 (path 4 -> 2 -> 1 -> 3, or 5 -> 2 -> 1 -> 3)

Idea: the diameter *through* any given node equals the sum of its left and right subtree heights - and the overall answer is the best such sum seen at *any* node, not necessarily the root (that's why the path "does not need to pass through the root"). Rather than computing heights and diameter as two separate traversals, compute them together in one pass: the height function (identical to 02's max_depth) updates a **diameter** variable as a side effect every time it's called, capturing "the best diameter seen so far through the current node" while still returning the height needed by the caller's own diameter calculation one level up.

Code

```
from tree_node import build_tree

def diameter_of_binary_tree(root):
    diameter = 0

    def height(node):
        nonlocal diameter
        if not node:
            return 0
        left_h = height(node.left)
        right_h = height(node.right)
        diameter = max(diameter, left_h + right_h)
        return 1 + max(left_h, right_h)

    height(root)
    return diameter

if __name__ == "__main__":
    tests = [
        ([1, 2, 3, 4, 5], 3),
        ([1, 2], 1),
        ([], 0),
        ([1], 0),
```

```

]

for i, (inp, expected) in enumerate(tests, 1):
    got = diameter_of_binary_tree(build_tree(inp))
    status = "PASS" if got == expected else "FAIL"
    print(f"Test {i}: {status} (got {got}, expected {expected})")

```

07. Balanced Binary Tree

07_balanced_binary_tree.py

Explanation

Problem: Given the root of a binary tree, determine if it is height-balanced: for every node, the heights of its left and right subtrees differ by at most 1.

Source : LeetCode 110 - Balanced Binary Tree

Example:

Input: 3

/\

9 20

/\

15 7

Output: True

Idea: the naive approach recomputes height from scratch at every node while also checking balance, giving $O(n^2)$ in the worst case (a skewed tree). Better: reuse 02's height computation, but make it return -1 as a sentinel the instant any subtree below is found unbalanced, and propagate that -1 upward immediately without doing any more work. Every node's height is then computed exactly once ($O(n)$ total), and an early -1 short-circuits the rest of the tree once imbalance is already known - no need to check nodes above it.

Code

```

from tree_node import build_tree

def is_balanced(root):
    def height(node):
        if not node:
            return 0

        left_h = height(node.left)
        if left_h == -1:
            return -1

        right_h = height(node.right)
        if right_h == -1:
            return -1

        if abs(left_h - right_h) > 1:
            return -1

        return 1 + max(left_h, right_h)

```

```

        return height(root) != -1

if __name__ == "__main__":
    tests = [
        ([3, 9, 20, None, None, 15, 7], True),
        ([1, 2, 2, 3, 3, None, None, 4, 4], False),
        ([], True),
        ([1], True),
    ]

    for i, (inp, expected) in enumerate(tests, 1):
        got = is_balanced(build_tree(inp))
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")

```

08. Validate Bst

08_validate_bst.py

Explanation

Problem: Given the root of a binary tree, determine if it is a valid binary search tree (every node's value strictly greater than all values in its left subtree and strictly less than all values in its right subtree - not just compared to its immediate children).

Source : LeetCode 98 - Validate Binary Search Tree

Example:

Input: 5

/ \

1 4

/ \

3 6

Output: False (4's left child 3 is fine locally, but 3 also sits in 5's right subtree and must be > 5, which it isn't - a check against immediate parents only would miss this)

Idea: checking each node against only its direct parent is the classic wrong approach - it misses violations further up the tree, as in the example above. Instead, carry a valid (low, high) range down through the recursion: the root can be anything, but each left child tightens the range's upper bound to the parent's value, and each right child tightens the lower bound. A node is valid only if its value falls strictly inside the range accumulated from *every* ancestor, not just its immediate parent.

Code

```

from tree_node import build_tree

def is_valid_bst(root):
    def validate(node, low, high):

```

```

        if not node:
            return True
        if not (low < node.val < high):
            return False
        return validate(node.left, low, node.val) and validate(node.right, node.val, high)

    return validate(root, float("-inf"), float("inf"))

if __name__ == "__main__":
    tests = [
        ([2, 1, 3], True),
        ([5, 1, 4, None, None, 3, 6], False),
        ([], True),
        ([1], True),
        ([5, 4, 6, None, None, 3, 7], False),
    ]

    for i, (inp, expected) in enumerate(tests, 1):
        got = is_valid_bst(build_tree(inp))
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")

```

09. Lowest Common Ancestor Bst

09_lowest_common_ancestor_bst.py

Explanation

Problem: Given the root of a BST and two nodes p and q known to exist in it, find their lowest common ancestor (the deepest node that has both p and q as descendants; a node can be its own descendant).

Source : LeetCode 235 - Lowest Common Ancestor of a Binary Search Tree

Example:

BST: 6

/ \

2 8

/ \ \

0 4 7 9

/ \

3 5

LCA(2, 8) = 6

LCA(2, 4) = 2

Idea: unlike the general-tree version (10_lowest_common_ancestor_binary_tree), this can exploit the BST ordering property directly, without exploring both subtrees. Starting at the root: if both p and q are smaller than the current node, the LCA must be in the left subtree (both are there); if both are larger, it must be in the right subtree. The moment they're *not* both on the same side, the current node is exactly the split point - and therefore the LCA - since that's the deepest node from which p and q diverge. No recursion into both branches needed, just a single walk down.

Code

```
from tree_node import build_tree, find_node

def lowest_common_ancestor(root, p, q):
    curr = root
    while curr:
        if p.val < curr.val and q.val < curr.val:
            curr = curr.left
        elif p.val > curr.val and q.val > curr.val:
            curr = curr.right
        else:
            return curr
    return None

if __name__ == "__main__":
    tree = build_tree([6, 2, 8, 0, 4, 7, 9, None, None, 3, 5])
    tests = [
        (2, 8, 6),
        (2, 4, 2),
        (0, 5, 2),
        (7, 9, 8),
    ]

    for i, (p_val, q_val, expected_val) in enumerate(tests, 1):
        p, q = find_node(tree, p_val), find_node(tree, q_val)
        got = lowest_common_ancestor(tree, p, q)
        got_val = got.val if got else None
        status = "PASS" if got_val == expected_val else "FAIL"
        print(f"Test {i}: {status} (LCA({p_val}, {q_val}) -> got {got_val}, expected {expected_val})")
```

10. Lowest Common Ancestor Binary Tree

10_lowest_common_ancestor_binary_tree.py

Explanation

Problem: Given the root of a general binary tree (no BST property) and two nodes p and q known to exist in it, find their lowest common ancestor.

Source : LeetCode 236 - Lowest Common Ancestor of a Binary Tree

Example:

Tree: 3

/ \

5 1

/ \ \

6 2 0 8

/ \

7 4

LCA(5, 1) = 3

LCA(5, 4) = 5

Idea: without BST ordering, there's no shortcut for which side to search - both subtrees must be explored. Recurse into left and

right; a call returns the node itself the moment it *is* p or q (no need to look further down that branch - a node can be its own ancestor). If both the left and right recursive calls come back non-None, that means p was found on one side and q on the other, so the *current* node is exactly where their paths converge - the LCA. If only one side returns non-None, both p and q must be in that same subtree, so pass that result upward unchanged.

Code

```
from tree_node import build_tree, find_node

def lowest_common_ancestor(root, p, q):
    if not root or root is p or root is q:
        return root

    left = lowest_common_ancestor(root.left, p, q)
    right = lowest_common_ancestor(root.right, p, q)

    if left and right:
        return root
    return left or right

if __name__ == "__main__":
    tree = build_tree([3, 5, 1, 6, 2, 0, 8, None, None, 7, 4])
    tests = [
        (5, 1, 3),
        (5, 4, 5),
        (6, 4, 5),
        (0, 8, 1),
    ]

    for i, (p_val, q_val, expected_val) in enumerate(tests, 1):
        p, q = find_node(tree, p_val), find_node(tree, q_val)
        got = lowest_common_ancestor(tree, p, q)
        got_val = got.val if got else None
        status = "PASS" if got_val == expected_val else "FAIL"
        print(f"Test {i}: {status} (LCA({p_val}, {q_val}) -> got {got_val}, expected {expected_val})")
```